Лабораторна робота №5

Динамічна ідентифікація типів

Виконала студентка групи МІТ-41

Пась Олександр

Хід виконання

Варіант 3: class Споруда.

3. Описати клас, забезпечивши наявність:

• 4-х властивостей різних типів, одна з яких мала б тип колекції;

• 3-х методів;

• 2-х конструкторів.

```
2 references
public string Name { get; set; }
3 references
public int YearBuilt { get; set; }
2 references
public double Height { get; set; }
4 references
public List<string> Purposes { get; set; }
```

рис. 5.1 - властивості різних типів

```
0 references
public void AddPurpose(string purpose)
{
    if (!string.IsNullOrEmpty(purpose))
        Purposes.Add(purpose);
}

0 references
public bool RemovePurpose(string purpose)
{
    return Purposes.Remove(purpose);
}

0 references
public int GetAge()
{
    return DateTime.Now.Year - YearBuilt;
}
```

рис. 5.2 - методи

```
public Building()
{
    Name = "Невідома споруда";
    YearBuilt = 2000;
    Height = 0;
    Purposes = new List<string>();
}

1 reference
public Building(string name, int yearBuilt, double height, List<string> purposes)
{
    Name = name;
    YearBuilt = yearBuilt;
    Height = height;
    Purposes = purposes ?? new List<string>();
}
```

рис. 5.3 - конструктори

4. Описати метод, який би виводив перелік всіх властивостей (разом з типом та значенням) об’єкта описаного класу у TreeView.

```
private void DisplayProperties(object obj)
{
    Type type = obj.GetType();

    // Кореневий вузол
    TreeNode root = new TreeNode($"Клас: {type.Name}");
    treeViewProperties.Nodes.Add(root);

    foreach (PropertyInfo prop in type.GetProperties())
    {
        object value = prop.GetValue(obj);

        if (value is System.Collections.IEnumerable enumerable && !(value is string))
        {
            TreeNode collectionNode = new TreeNode($"{prop.Name} ({prop.PropertyType.Name})");

            foreach (var item in enumerable)
            {
                collectionNode.Nodes.Add($"({item.GetType().Name}) {item}");
            }

            root.Nodes.Add(collectionNode);
        }
        else
        {
            root.Nodes.Add($"{prop.Name} ({prop.PropertyType.Name}) = {value}");
        }
    }

    treeViewProperties.ExpandAll();
```

рис. 5.4 - метод

Метод DisplayProperties показує всі властивості об’єкта у TreeView. Спочатку створюється кореневий вузол із назвою класу. Потім перебираються всі властивості об’єкта через рефлексію: якщо властивість — колекція, її елементи додаються як підвузли; якщо ні — показується назва, тип і значення властивості. Наприкінці дерево розгортається, щоб було видно всі вузли.

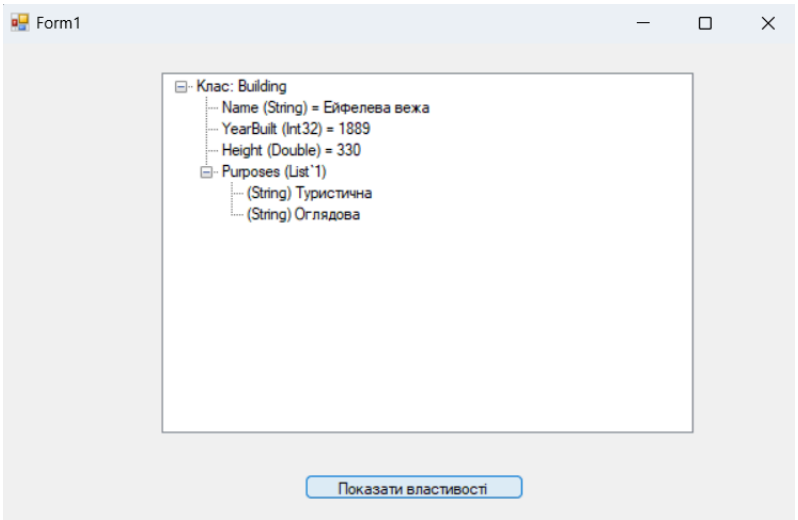

рис. 5.5 - результат

Висновок: У цій лабораторній роботі ми створили клас Building із різними типами властивостей, методами для роботи з ними та конструкторами. Також реалізували метод DisplayProperties, який за допомогою рефлексії відображає всі властивості об’єкта у TreeView, включно з колекціями. Це дозволяє динамічно переглядати стан об’єкта та перевіряти роботу його методів.